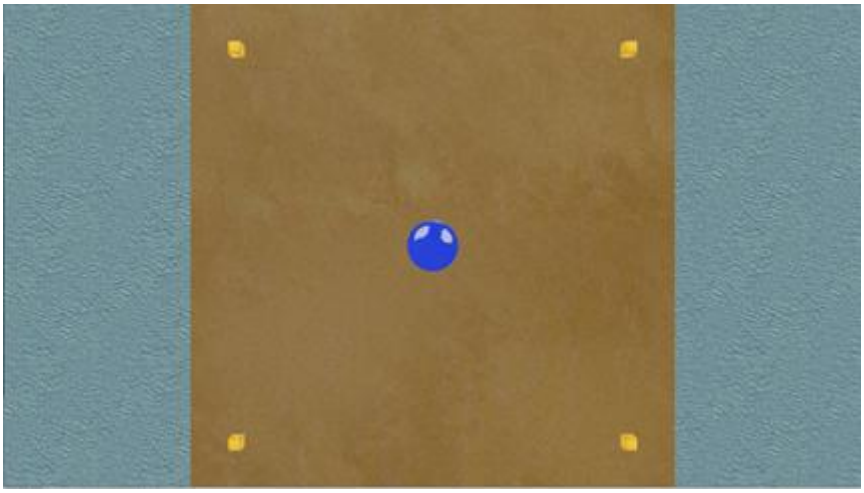
CO1035: Game Concepts
Assessment 1 - Spheres.io
Implementation Guide

This document is not meant to serve as an Assessment Brief or its replacement; it is simply meant to help guide those who seek guidance through implementing the features specified in the Assessment Brief. **Make sure you read and understand the brief before reading this document**. Those that do not seek guidance can ignore this document completely, or turn towards it only in time of dire need. Note that it is completely acceptable for your implementation approach to differ from what is described in this document, **as long as it meets the specifications of the Assessment Brief.**

1. BASIC LAYOUT

Setting up the basic scene requires loading the meshes provided, creating appropriate models, and positioning the models as specified by the brief. You should have completed similar tasks in every lab since Lab1 of this module, for example, see your solution for section 2 of Lab5 or Lab5's walkthrough video. On implementing the basic layout as specified, your scene should look like the image below.



2. MILESTONE 1: THIRD CLASS (UPTO 49%)

You must have implemented the basic scene as specified by the brief to get full marks for implementing features in this section. The features in this milestone can be implemented as described below, and on completion, the game should look similar to the video that follows.

- To implement WASD controls for moving the sphere and arrow controls for the camera use **Lab2_Controls_Project** and **Lab3_CameraControl_Project** as references.
- To check whether the sphere has moved into the water, use **Lab2_TestingVariables_Project** as a reference.
- To implement Paused and Playing states, you can use a boolean variable to store this binary state (use **Lab2_TestingVariables_Project** as a reference). However, if you are implementing features up to MILESTONE 3 or higher, you must utilise an enumeration to track the states of the game i.e. Playing, Paused and GameOver, use **Lab5_SphereStates_Project** as a reference. For this milestone, the game should just end when it enters GameOver state.
- To implement box pickup by the sphere, you have to calculate the distance between the sphere and the all the cubes in the scene at every frame by doing the following for each cube in the scene:
    1. Calculate the vector between the sphere and the cube
    2. Calculate the length of the vector, this is the distance between the sphere and the cube

3. If the distance is less than the sum of the sphere radius (10) and half of the cube's side length (2.5), then the cube should be picked up.
4. Make the cube invisible, you can do that by changing its position to be out of view.

Vectors are introduced in Lecture 6 - Vectors, you can use the formula introduced to calculate the vector between the sphere and each cube, and their lengths. You can also use **Lab7_Drone_Project** as a reference. However note that **Lab7_Drone_Project** goes on to calculate direction vectors, for this assessment you are only interested in calculating the length of a vector and comparing it to some distance.

3. MILESTONE 2: LOWER SECOND CLASS (UPTO 59%)

You must have implemented the basic scene as specified by the brief, and **MILESTONE 1** to get full marks for implementing features in this section. The features in this milestone can be implemented as described below, and on completion, the game should look similar to the video that follows.

- To implement an isometric camera on pressing "2", listen for a key hit on the 2 key, and update the position and rotation of the camera as specified by the brief. It is important that you first reset the orientation of the camera before making these updates (use **camera->ResetOrientation()**). If the isometric camera is active, listen for key hit on the 1 key to update the position and rotation back to the top-down camera specified in the basic scene spec, again remember to reset the orientation before making the updates. Also remember that the arrow controls should not work on the camera if it is isometric, therefore you may have to use states to track the active camera, **Lab5_SphereStates_Project** can be used as a reference for this.
- To implement point keeping, declare a variable for storing points and increment its value every time a box is picked up i.e. every time the distance between the sphere and a box is small enough. To display the points on the screen, use **Lab4_KeepingScore_Project** as a reference
- To implement sphere resize, you need to keep track of the number of cuved picked up by the sphere (e.g. using a counter or even using the score). If 4 spheres are picked, use the **sphere->Scale(x)** to increase the scale of the sphere to 1.2 its initial size. To ensure parts of the sphere are not below the floor after reseize, the y-position of the sphere must also be scaled by 1.2.

4. MILESTONE 3: UPPER SECOND CLASS (UPTO 59%)

You must have implemented the basic scene as specified by the brief, **MILESTONE 1** and **MILESTONE 2** to get full marks for implementing features in this section. The features in this milestone can be implemented as described below, and on completion, the game should look similar to the video that follows.

- To implement an array for storing 12 cube models, use **Lab6_Vectors_Project** as a reference. Remember, you now need to scale the cube to 1.2 its initial size every time it picks up 4 cubes. You may also notice a reduction in the speed of your game due to the additional cubes and the increased number of collision check calculations, to remove this effect, start utilising variable timing to move and rotate the sphere, use **Lab7_Drones_Project** as a reference.
- To implement random positions for the cubes, you must utilise some random function to generate random x and z positions for each cube (y-position should be the same for all cubes), these random positions must be restricted to values that will place the cube on the floor. The values should be generated and used to position the cubes in the same loop that creates them and stores them in the array implemented above. You can use **CO1404's Lab 3** as a reference for generating random numbers or look up C++'s rand() function.
- To implement sphere to box collision detection, use **Lab8_BoxCollisions_Project** as a reference (this will be available to you on 17/12/2021.

- For this milestone, you must track the four game states, Playing, Paused, GameOver and GameWon, using an enumeration; use **Lab5_SphereStates_Project** as a reference. Movement should only be allowed in the Playing state; everything should be paused in the Pause state, and appropriate messages should be displayed in GameOver and GameWon states.

5. MILESTONES 4 & 5: FIRST CLASS (70%+)

You must have implemented the basic scene as specified by the brief, **MILESTONE 1**, **MILESTONE 2** and **MILESTONE 3** to get full marks for implementing features in this section. The features in this milestone can be implemented as describe below, and on IMPLEMTING , **MILESTONE 4** features the game should look similar to the video that follows. No video is provided for **MILESTONE 5** as the milestone allows creative flexibility.

- To implement the separation between spawned cubes, use a while loop to keep generating random positions for each cube to be spawned until the position generated is more than 10 units away from all other cubes spawned and the sphere. This means that for each cube to be spawned, you need to: generate a random x and z values to crete a random position, calculate the vector between the random position and the position of all cubes that have been spawned, calculate the length of the vector, while the length is less than 10 repeat the process.
  Revisit **CO1404's Lab4 and Lab5** sheets and solutions for a reminder on while loops, you can think of this task as using a while loop to validate the random position generated.
- To implement random respawn, you will have to use the same process described above to find create a new position for a cube that is picked up by the sphere (Instead of hiding it as done in previous milestones)
- To implement "Hpper Mode", you will need to utilise states to track whether the sphere is in regular or hyper mode. In hyper mode, the sphere skin should change, and in addition to the sphere to box collision detection for box pickup, you also need to perform distance checks between all cubes and the sphere and have every cube within 50 units of the sphere moved towards the sphere. A cube moving towards the sphere does not get picked up until it gets close enough to the shere for the sphere to box collision to come into effect.
- Hper mode should last five seconds, the sphere should return to its regulaar state after that.To implement this, remember that the Timer() method returns the time that has passed since the last frame was rendered (in milliseconds).
- The NPC sphere should move in some intelligent or at least random manner, be able to pick up cubes like the player sphere, increase in size after every 4 pickups, should not move off the floor, and should be able to enter Hyper Mode. Use sphere to sphere collision detection to check for collisions between the two spheres and determine the resolution based on their points, as specified by the brief.